Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Mahmoud Ayedi

Année Universitaire : 2025-2026

1 Mise en place de l'environnement

Après avoir créé la structure de dossiers demandée, nous avons initialisé un environnement virtuel Python pour isoler les dépendances du projet. .

Les librairies nécessaires ont été installées en une seule commande, puis le fichier `requirements.txt` a été généré automatiquement avec `pip freeze`. Le projet a été initialisé avec Git et un fichier `.gitignore` configuré pour exclure le dossier `venv/`, les fichiers `__pycache__/` et le dossier `data/raw/` contenant les données brutes.

2 Exploration des données

2.1 Chargement et premières observations

Le dataset a été chargé dans un notebook Jupyter `notebooks/01_exploration.ipynb`. La première observation avec `df.info()` a révélé que le dataset contient **4 372 clients** et **52 colonnes** de types variés (int64, float64, str).

L'appel à `df.isnull().sum()` a permis d'identifier deux colonnes avec des valeurs manquantes :

Colonne	Valeurs manquantes	Pourcentage
Age	1 311	29.99%
AvgDaysBetweenPurchases	79	1.81%

2.2 Détection des valeurs aberrantes

L'appel à `df['SupportTicketsCount'].value_counts()` a révélé la présence de valeurs **999** (87 occurrences) et **-1** (43 occurrences) qui sont des codes d'erreur. De même, `SatisfactionScore` contient des valeurs **99** (114 occurrences) et **-1** (115 occurrences).

2.3 Analyse de la variable cible

La distribution du Churn a été visualisée avec `sns.countplot()` :

- Churn = 0 (Fidèles) : 2 918 clients — **66.7%**
- Churn = 1 (Partis) : 1 454 clients — **33.3%**

Le déséquilibre modéré (ratio 2 :1) a été pris en compte via `stratify=y` lors du split train/test pour conserver les proportions dans les deux ensembles.

2.4 Analyse des corrélations

Une heatmap de corrélation a été générée avec `sns.heatmap(df.corr())` pour identifier les relations entre features. L'analyse `df.corr()['Churn'].abs().sort_values(ascending=False)` a révélé que certaines colonnes avaient une corrélation anormalement élevée avec la cible — ce qui a conduit à la détection du data leakage décrite en section 7.

3 Préparation des données — `src/preprocessing.py`

3.1 Correction des valeurs aberrantes

Nous avons utilisé la méthode `replace()` de pandas pour remplacer les codes d'erreur par `np.nan`, ce qui permet de les traiter ensuite avec l'imputation :

```
df['SupportTicketsCount'] = df['SupportTicketsCount'].replace({999: np.nan, -1: np.nan})
df['SatisfactionScore']   = df['SatisfactionScore'].replace({99: np.nan, -1: np.nan})
```

3.2 Parsing de RegistrationDate

La colonne `RegistrationDate` était de type `str` avec des formats inconsistants. Nous avons utilisé `pd.to_datetime()` avec `format='mixed'` pour gérer tous les formats en une seule passe, puis extrait trois nouvelles features utiles :

```
df['RegistrationDate'] = pd.to_datetime(df['RegistrationDate'],
                                        format='mixed', dayfirst=True,
                                        errors='coerce')
df['RegYear']    = df['RegistrationDate'].dt.year
df['RegMonth']   = df['RegistrationDate'].dt.month
df['RegWeekday'] = df['RegistrationDate'].dt.weekday
df = df.drop(columns=['RegistrationDate'])
```

3.3 Feature engineering sur LastLoginIP

L'adresse IP brute n'est pas exploitable directement. Nous avons créé une feature binaire `IsPrivateIP` qui vaut 1 si l'IP commence par `192.168`, `10.` ou `172.` (réseaux privés), et 0 sinon :

```
def is_private_ip(ip):
    return int(str(ip).startswith(('192.168', '10.', '172.')))
df['IsPrivateIP'] = df['LastLoginIP'].apply(is_private_ip)
df = df.drop(columns=['LastLoginIP'])
```

3.4 Suppression des colonnes inutiles

La colonne `NewsletterSubscribed` a été supprimée car elle ne contient qu'une seule valeur (`"Yes"`) — sa variance est nulle, elle n'apporte aucune information au modèle. `CustomerID` a également été supprimé car c'est un identifiant sans valeur prédictive. La colonne `ChurnRiskCategory` a été supprimée dès cette étape car elle cause un data leakage direct (corrélation 0.88 avec Churn).

3.5 Imputation KNN

Pour remplacer les valeurs manquantes, nous avons utilisé `KNNImputer(n_neighbors=5)` de scikit-learn. Cet imputer remplace chaque valeur manquante par la moyenne des 5 clients les plus proches dans l'espace des features. La variable cible `Churn` a été exclue de l'imputation :

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
```

Résultat : **0 valeur manquante** après imputation.

3.6 Encodage des variables catégorielles

Trois stratégies d'encodage ont été appliquées selon le type de chaque variable :

- **Encodage ordinal** pour les variables avec un ordre logique (`SpendingCategory`, `LoyaltyLevel`, `BasketSizeCategory`, `AgeCategory`) : on assigne manuellement des entiers respectant l'ordre (ex : Low=0, Medium=1, High=2, VIP=3).
- **One-Hot Encoding** via `pd.get_dummies()` pour les variables nominales sans ordre (`RFMSegment`, `CustomerType`, `Region`, `Gender`...) : chaque valeur devient une colonne binaire.
- **Label Encoding** pour `Country` (37+ valeurs) : `pd.get_dummies()` aurait créé trop de colonnes.

Le dataset passe de **52 à 76 colonnes** après encodage.

4 Séparation Train/Test — src/train_test_split.py

Nous avons utilisé `train_test_split()` avec `stratify=y` pour conserver les proportions de churn dans les deux ensembles. La normalisation `StandardScaler` a été appliquée **après** le split — c'est un point critique :

Pourquoi normaliser après le split ?

Si on normalise avant le split, les statistiques (moyenne, écart-type) du jeu de test influencent la normalisation du jeu d'entraînement. Le modèle "voit" indirectement les données de test pendant l'entraînement — c'est du **data leakage**. La règle est : `fit_transform` sur X_train uniquement, puis `transform` (sans fit) sur X_test.

```
scaler = StandardScaler()
X_train[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test[numeric_cols]  = scaler.transform(X_test[numeric_cols])
```

Résultat : **3 497 clients en train** (80%) et **875 clients en test** (20%).

5 Réduction de dimension — ACP

L'ACP a été appliquée dans `src/clustering.py` pour visualiser les clusters en 2D. Nous avons utilisé `PCA(n_components=2)` de scikit-learn sur X_train après normalisation :

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_train)
print(f"Variance expliquee : {pca.explained_variance_ratio_.sum():.1%}")
# >> Variance expliquee : 17.7%
```

Les 2 premières composantes n'expliquent que **17.7%** de la variance totale. Ce résultat est attendu pour un dataset à forte dimensionnalité (76 features) — l'information est répartie sur de nombreux axes. La visualisation 2D reste utile pour observer la séparation qualitative des clusters même si elle ne capture pas toute la structure.

6 Clustering K-Means — src/clustering.py

6.1 Recherche du K optimal

Pour déterminer le nombre de clusters optimal, nous avons combiné deux méthodes implémentées dans une boucle de K=2 à K=8 :

```
for k in range(2, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_train)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_train, km.labels_))
```

Les résultats obtenus :

K	Inertie	Score Silhouette
2	244 727	0.089
3	228 541	**0.090**
4	218 564	0.086
5	207 650	0.088

Bien que K=3 donne le meilleur score silhouette, l'analyse des clusters a révélé qu'il produisait un micro-cluster de seulement **2 clients** avec des valeurs extrêmes (`ReturnRatio` = 3.2). Nous avons donc retenu **K=2** qui donne une segmentation plus robuste et interprétable.

6.2 Interprétation des clusters

Cluster	Taille	Fréquence moy.	Profil
0	1 917 (55%)	-0.35	Peu actifs, faibles dépenses, diversité produits faible — à risque churn
1	1 580 (45%)	+0.42	Actifs, fortes dépenses, diversité produits élevée — fidèles

La colonne `Cluster` (0 ou 1) a été ajoutée au dataset comme feature supplémentaire avant la modélisation.

7 Modélisation — `src/train_model.py`

7.1 Détection et correction du data leakage

Lors des premiers tests, les trois modèles obtenaient un score parfait (Accuracy = AUC = **1.0000**). Pour diagnostiquer ce problème, nous avons calculé la corrélation de chaque feature avec `Churn` dans X_train :

```
correlations = X_train.corrwith(y_train).abs().sort_values(ascending=False)
```

Puis nous avons testé l'importance des features avec un Random Forest pour identifier lesquelles le modèle utilisait massivement. La cause était que plusieurs colonnes avaient été construites à partir de la variable cible dans ce dataset synthétique (`ChurnRiskCategory` avec corrélation 0.88, `CustomerType_Perdu` à 0.70, etc.).

Ces colonnes ont été supprimées, réduisant les features de 76 à **63 features légitimes**.

7.2 Entraînement des trois modèles

Trois algorithmes ont été entraînés et comparés :

```
# Regression Logistique
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

# Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train, y_train)
```

7.3 Résultats et comparaison

Modèle	Accuracy	AUC-ROC	Precision	Recall
Régression Logistique	0.8869	0.9455	0.83	0.84
Random Forest	0.8869	0.9298	0.91	0.73
Gradient Boosting	**0.9269**	**0.9624**	**0.94**	**0.84**

Le **Gradient Boosting** est sélectionné comme meilleur modèle car il obtient le meilleur AUC-ROC (0.9624) et la meilleure accuracy (92.7%), tout en maintenant un recall de 84% sur les churners — ce qui signifie qu'il détecte 84% des clients qui vont réellement partir.

Les modèles ont été sauvegardés avec `joblib.dump(model, 'models/nom_modele.pkl')` pour être réutilisés sans ré-entraînement.

7.4 Optimisation des hyperparamètres

`GridSearchCV` a été utilisé pour tester toutes les combinaisons d'hyperparamètres via validation croisée à 5 folds :

```
param_grid = {
    'n_estimators':  [100, 200],
    'max_depth':     [3, 5],
    'learning_rate': [0.05, 0.1],
    'subsample':     [0.8, 1.0]
}
grid_search = GridSearchCV(GradientBoostingClassifier(random_state=42),
                           param_grid, cv=5, scoring='roc_auc', n_jobs=-1)
grid_search.fit(X_train, y_train)
```

8 Prédiction — src/predict.py

Le script `predict.py` permet de faire des prédictions sur de nouveaux clients sans relancer tout le pipeline. Deux fonctions principales ont été implémentées :

- `predict_single(client_data)` : prend un dictionnaire de features et retourne la prédiction, la probabilité et le niveau de risque pour un seul client.
- `predict_batch(csv_path)` : charge un fichier CSV de nouveaux clients, génère les prédictions pour tous et sauvegarde les résultats dans `data/processed/predictions.csv`.

Pour aligner les features du nouveau client avec celles attendues par le modèle, nous créons un DataFrame avec toutes les colonnes à zéro, puis on renseigne uniquement les valeurs disponibles :

```
df = pd.DataFrame(np.zeros((1, len(model_cols))), columns=model_cols)
for key, value in client_data.items():
    if key in df.columns:
        df[key] = float(value)
```

Le test de validation sur deux clients réels du jeu de test a confirmé le bon fonctionnement :

Client	Vrai label	Prédiction	Probabilité churn
Client fidèle réel	Fidèle	Fidèle	6.5%
Client churner réel	Churné	Churné	96.9%

9 Déploiement Flask — app/app.py

L'application Flask expose une route `POST /predict` qui reçoit les données d'un client en JSON, reconstruit le vecteur de features complet, charge le modèle Gradient Boosting sauvegardé et retourne la prédiction.

Pour résoudre le problème de chemins relatifs (le script étant dans `app/` mais les modèles dans `models/`), nous avons utilisé `os.path.dirname()` pour construire des chemins absolus :

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
model = joblib.load(os.path.join(BASE_DIR, 'models', 'gradient_boosting.pkl'))
```

L'interface HTML/CSS/JavaScript permet à l'utilisateur de saisir les caractéristiques d'un client via un formulaire, d'envoyer une requête AJAX vers `/predict` et d'afficher le résultat dynamiquement sans rechargement de page.

10 Conclusion

Ce projet nous a permis d'implémenter une chaîne complète de data science sur un dataset réel de 4 372 clients e-commerce. Les principaux apprentissages techniques sont :

1. **L'ordre des opérations est critique** : normaliser avant le split cause un data leakage subtil mais réel.
2. **Le data leakage peut être invisible** : des scores parfaits (1.0) sont le signe d'un problème, pas d'un succès.
3. **L'interprétation métier est indispensable** : identifier qu'une colonne est construite à partir de la cible nécessite de comprendre les données, pas seulement les chiffres.
4. **Le Gradient Boosting** s'impose comme le meilleur modèle avec **Accuracy 92.7%** et **AUC-ROC 96.2%** sur 63 features légitimes.

Sur le plan métier, le modèle déployé permet à l'entreprise de détecter **84% des clients qui vont partir** avant qu'ils ne le fassent, laissant le temps de mettre en place des actions de rétention ciblées.